Revolutionizing Cloud Security with Programmable Frameworks: a Novel Approach

Alessandro Carrega*
* Department of Electrical, Electronic and Telecommunications Eng., and Naval Architecture (DITEN)
University of Genoa (UniGe), Italy {*name*}.{*surname*}@unige.it

2024 IEEE Future Networks World Forum (FNWF) | 979-8-3503-7949-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/FNWF63303.2024.11028857

***Abstract*—We present a novel framework for the management of a multi-tier architecture, where a common, programmable, and pervasive context fabric feeds a powerful set of multi-vendor detection and analysis algorithms (business logic). The challenge is deep visibility over multiple software components by real-time collection of massive events from a multiplicity of capillary sources, while maintaining essential properties such as forwarding speed, scalability, autonomy, usability, fault tolerance, resistance to compromises, and responsiveness. The ambition is to support better and more reliable situational awareness by inter- and intra-domain data correlation in both space and time, in order to timely detect and respond even the more sophisticated multi-vector and interdisciplinary cyberattacks. The Context Broker (CB) is the logical component to manage the security context. We define the security context as the set of information, data, and measurements that describe the service and can be used for security-related purposes. The Local Control Plane (LCP) gives the CB access to the configuration of agents. We performed an evaluation of the CB Manager (CB-Man) and LCP considering different scenarios and workloads. The goal is to verify the robustness and the reliance of these two components in different execution scenarios.**

***Index Terms*—Security, Framework, Programmability, Cloud**

I. Introduction

ASTRID (AddreSsing ThReats for virtualIseD services) is a multi-tier architecture, where a common, programmable, and pervasive context fabric feeds a powerful set of multi-vendor detection and analysis algorithms (business logic). On the one hand, the challenge is deep visibility over multiple software components by real-time collection of massive events from a multiplicity of capillary sources, while maintaining essential properties such as forwarding speed, scalability, autonomy, usability, fault tolerance, resistance to compromises, and responsiveness [1].

On the other hand, the ambition is to support better and more reliable situational awareness by inter- and intra-domain data correlation in both space and time, in order to timely detect and respond even the more sophisticated multi-vector and interdisciplinary cyberattacks [2]. The Context Broker (CB) is the logical component to manage the security context. We define the security context as the set of information, data, and measurements that describe the service and can be used for security-related purposes. The scope includes the description of the service components (namely, software deployed in each virtual function) and its topology (network links and communication channels), as well as operational data from the execution of the service (logs, system metrics, measurements, events, software traces) [3]. The CB Manager (CB-Man) function implements a Representational State Transfer (REST)-Application Programming Interface (API) that provides a uniform access interface to multiple components and the information stored in the Context Broker (CB)'s database:

a) detection and monitoring agents deployed in virtual functions: location, capability, properties;
b) external service orchestrator: service topology, run-time parameters;
c) security services: capability, parameters; and
d) data stored in the CB, as reported by the running agents: retrieval, queries.

The Local Control Plane (LCP) gives the CB access to the configuration of agents. It is designed to support multiple configuration methods (through configuration file, command line, REST API) but to remain unaware of specific protocols. The Local Control Plane (LCP) acts as the single point of contact for the CB and exposes the list of available agents.

The architecture includes a number of components to be deployed in the execution environment of Virtual Functions (VFs), and the internal structure of the CB. All components are organized in a data plane and a control plane; the management plane is not shown in the picture because it is entirely implemented by the external software orchestration tool. The overall design is based on the Elasticsearch, Logstash and Kibana (ELK) framework [4], a collection of open source projects for data acquisition, processing, and storage. It was originally composed of Elasticsearch [5], Logstash [6], and Kibana [7] (for which it was formerly known as ELK) and is now evolving to include additional components. Though these tools were originally conceived to work statically, i.e. with minimal or no possibility to change the configuration at run-time, we are implementing an additional dimension of programmability in the control plane [8], so to easily support the definition of new inspection and monitoring tasks at runtime [9].

In the last part, we performed an evaluation of the CB Manager (CB-Man) and LCP considering different scenarios and workloads. The goal is to verify the robustness and the reliance of these two components in different execution scenarios.

II. Context Broker

The CB is the logical component to manage the security context. We define the security context as:

> *Set of information, data, and measurements that describe the service and can be used for security-related purposes. The scope includes the description of the service components (namely, software deployed in each virtual function) and its topology (network links and communication channels), as well as operational data from the execution of the service (logs, system metrics, measurements, events, software traces).*

Conceptually, the CB implements three main logical functions:

Context Delivery which is the real-time collection of data and measurements generated by local agents according to streaming patterns. The internal delivery function is expected to make this data directly available to intended consumers (i.e., analytics and detection algorithms implemented over the ASTRID platform) and to store it internally.

Context Storage which keeps historical data for offline analysis. It includes the whole context, therefore encompassing both service topology, available agents, and data generated by them.

Context Abstraction describes the overall service topology, including available agents, their capabilities, and their current configuration. This information is used by the control logic to configure agents, so to collect data that are needed by specific algorithms. Analytics and detection algorithms can also use this information

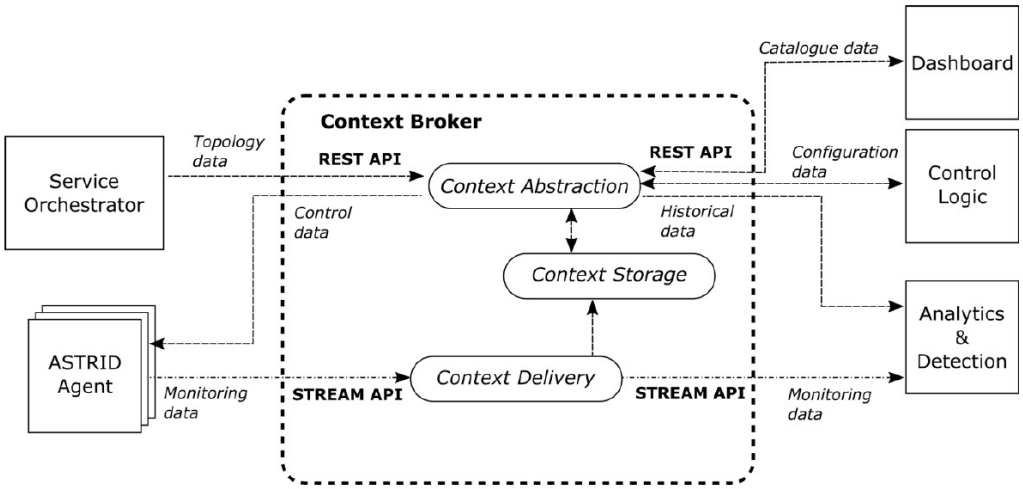

Figure 1. Conceptual model of the CB.

for correlating data and measurements based on the service topology, as well as to retrieve and make selective queries on historical data.

The first task for the CB is to manage the heterogeneity of sources and protocols, which is reflected in different data and control interfaces. The CB hides this heterogeneity and exposes a common context model to the other components in the security orchestrator through the "*Context Abstraction*" function, for discovering, configuring, and accessing the security context available from the execution environment. Though control and management of local agents is a common feature in existing Security Information and Event Management (SIEM) tools, automatic discovery of the service topology is currently not available, but this is very useful for cloud-based services, where the composition and topology are expected to autonomously change during the lifetime according to the evolving context [10].

The CB collects data from monitoring and inspection processes deployed in the execution environment ("*Context Delivery*" and "*Context Storage*"), hence implementing the required data channel envisioned by the ASTRID architectur.The CB hides the heterogeneity and asynchrony of the sources, feeds the analysis algorithms with the requested context, organizes historical data, and provides simple querying and fusion capabilities in data access. Given the very different semantics of the context data, the obvious choice is Not only SQL (NoSQL) databases. This allows defining different records for different sources, but also poses the challenge to identify a limited set of formats, otherwise part of the data might not be usable by some algorithms [11].

The flexibility in programming the execution environment is expected to potentially lead to a large heterogeneity in the kind and verbosity of data collected [12]. For example, some virtual functions may report detailed packet statistics (i.e., those at the external boundary of the service), whereas other functions might only report application logs. In addition, the frequency and granularity of reporting may differ for each virtual function. The definition of a (security) context model is therefore necessary for detection algorithms to know what could be retrieved (i.e., capabilities) and what is currently available, how often, with each granularity (i.e., configuration) [13]. The implementation of this model is the Security Context Abstraction, the homogeneous control interface that the CB offers for configuring and programming different data sources, by implementing the specific protocols, corresponding to the control channel in Figure 1. This is intended to be used during the design of analytics pipelines, to select and configure agents so that they produce data according to the format and content expected by the analytics and detection algorithms.

TABLE I
MAPPING OF CB FUNCTIONS TO THE SOFTWARE ARCHITECTURE.

FUNCTION	SOFTWARE COMPONENTS
Software Component	CB-Man
Context Abstraction	Kakfa
Context Manager	Logstash + Elasticsearch

Finally, Table I maps the logical functions of the CB to the software architecture.

III. CONTEXT BROKER MANAGER

The CB function implements a REST API that provides a uniform access interface to multiple components and the information stored in the CB's database:

- detection and monitoring agents deployed in virtual functions: location, capability, properties;
- external service orchestrator: service topology, run-time parameters;
- security services: capability, parameters;
- data stored in the CB, as reported by the running agents: retrieval, queries.

The CB-Man interface uses a single protocol but exposes different data models for each component, due to the large heterogeneity in the configuration properties of similar tools. The backend database for the CB-Man is Elasticsearch. Different indexes are used for data produced by agents, notifications sent by the security services, and context information. Context information includes the data models for agents, security services, and virtual services.

The main indexes used are types, catalogs, instances and data. Types and catalogs are managed by the system administrator to describe the objects available within the system: execution environments (virtual machines, containers), network links (Local Area Network (LAN), point-to-point, point-to-multipoint), agents, algorithms, and programs. Instances describe concrete instantiations of the objects, including virtual functions, agents, security services. They are typically created and managed by the Security Controller, based on the events reported by the external software orchestrator. Data is generated by

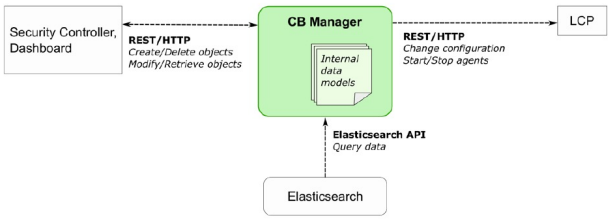

Figure 2. The CB-Man and its interfaces.

agents and can only be inserted through the data channel; the REST API only supports retrieval and queries on the data. This is mainly provided to hide the backend storage to the dashboard.

The CB-Man implements the following interfaces: REST APIs for interacting with the internal data models (Eastbound), consumed by the Security Controller and the Dashboard; consumer of the REST APIs for interacting with the LCP (Westbound); and consumer of Elasticsearch API for querying and retrieving objects in the datastore (Southbound).

The current implementation supports the following features: creation, modification, deletion, and retrieval of object types and instances; creation, modification, deletion, and retrieval of objects in the catalog; pushing agent configurations/programs to the LCP when the corresponding data model is updated; and authorization and access control on the Eastbound interface (HypterText Transfer Protocol (HTTP) and JSON Web Token (JWT) methods available).

IV. Local Control Plane

The LCP gives the CB access to the configuration of agents. It is designed to support multiple configuration methods (through configuration file, command line, REST API) but to remain unaware of specific protocols. The LCP acts as the single point of contact for the CB and exposes the list of available agents.

Standard beats from the Elastic stack (Filebeat, Metricbeat, Packetbeat) are configured by changing their configuration files, since there are no available APIs to dynamically program these agents [14]. They can change predefined options in each configuration file, according to the programming model exposed by the CB and restart the beat to apply the new configuration through the LCP. Indeed, the LCP is also used to invoke shell commands and scripts, which are used for starting/stopping the beat, so that no data is generated if not requested by any consumer in the framework. Cubes developed in the Polycube framework [15] are directly configured through their REST API, which is exposed by each cube. In this case, the LCP is mostly a proxy that forwards HTTP messages to the Polycube daemon. There is no need for shell commands in this case, since start/stop operations of cubes are directly managed by polycubed.

The LCP plays the role of local control point to give access to agents' configurations. It does not provide any monitoring data or event itself, but it is only used for configuring a heterogeneous set of agents. The LCP is deployed by the external service orchestrator, as any other local agent, and its configuration must include the list of installed agents. The LCP supports multiple configuration and control methods:

- editing of YAML Ain't Markup Language (YAML) files;
- forwarding of HTTP messages to the agent interface;
- invoking shell commands (mainly to start/stop/reload the agents).

The LCP can be defined as a dump configuration entity, which is used to get remote access to local agents. Indeed, it is not aware of the semantics of configuration messages: it just applies what the CB Manager provides. That means no drivers or plugins are

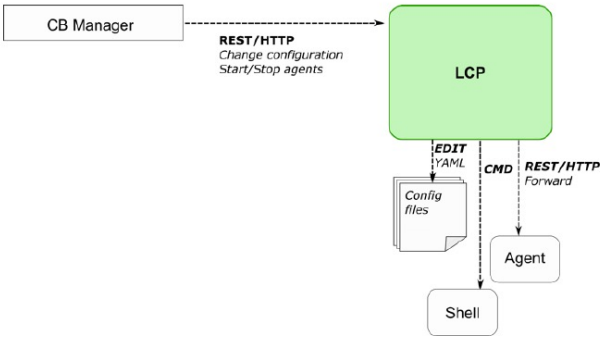

Figure 3. The LCP and its interfaces.

required for the agents; just the knowledge of what configuration method should be used. The LCP allows remote control even for those agents that do not provide a management interface. There is no need to open multiple ports on the firewall, and authentication and access control can be applied to a single entry point instead of many different interfaces. The advantage of implementing a LCP instead of relying on more general-purpose mechanisms (e.g., Secure Shell (SSH) connection) are the following:

- there is a consolidated practice for using implementing REST APIs on top of HTTP, and integration with data models is rather straightforward;
- parsing HTTP messages and applying access control is simpler than with a plain SSH connection;
- the local configuration agent might be used to apply default and fallback configurations in case the centralized platform is not reachable.

Once activated, the LCP is able to respond to connection requests from the CB-Manager. It can interact with different CB-Man instances at the same time. The CB-Man sends periodic heartbeating messages to verify the connectivity to the LCP. The heartbeating messages are also used to periodically renew the security credentials. The LCP then uses the most appropriate method to apply incoming configuration requests. The LCP is explicitly designed to interact with the CB-Manager, and no other kinds of usage are expected; hence, the REST requests must be made only from the CB-Man. However, for development and test purposes, common tools for testing REST interfaces can be used.

The LCP implements the following interfaces: REST-APIs for control of local agents (Eastbound), consumed by the CB-Man, file system access to edit files (Southbound); shell command execution to invoke management actions on local agents (Southbound); and HTTP client to forward messages to the interfaces of local agents (Southbound).

The current implementation supports the following features: editing of configuration files written with YAML syntax; forward of configuration messages to a local Polycube instance; execution of any command on a local shell; and authentication on the Eastbound interface.

V. Performance Evaluation

This section describes the performance evaluation of the CB-Man and LCP components. We consider 3 different scenarios in order to evaluate these two components varying the number of the HTTP requests and to verify a correct operation. Table II shows a short summary of the tests done. Each scenario requires a series of HTTP requests involving different entities (execution environment, network link, connection, etc.).

TABLE II
DIFFERENT SCENARIOS FOR THE PERFORMANCE EVALUATION.

SCENARIO	STEPS
Service Topology	Get all the data from various endpoints to obtain the info related to the service topology.
Execution Environment Setup	Add a new entry in the execution environment and check if the LCP is correctly connected.
Agent Management (Action)	Add a new agent instance with info from the catalog and execute the start and stop actions.
Agent Management	Add a new agent instance with info from the catalog, execute the start action, update a parameter, and finally execute the restart action.

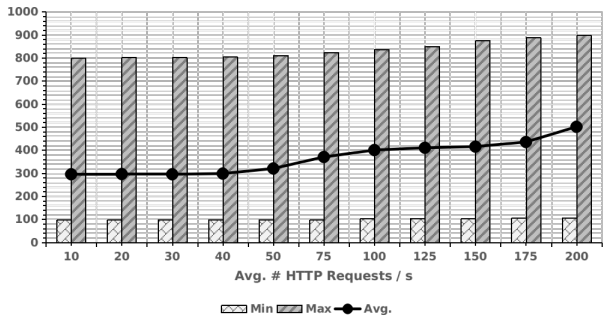

Figure 4. Response Time (in ms) varying the number of the single HTTP requests for the service topology scenario.

For this reason, we measured the statistics (average, min, and max) of the response time (in ms) considering the single HTTP requests and the flow composed of a group of HTTP requests required to complete the scenario. For example, the service topology scenario requires 12 different HTTP requests in order to obtain the needed info. The entire evaluation was done using Apache JMeter. The source code is available in the GitHub repositories of the project[1]. For each scenario we run different tests varying the number of HTTP requests per second from 10 to 100. Hence, considering that each scenario requires from 5 to 10 HTTP requests, the number of requests to complete the scenario is between 1 and 20.

VI. SERVICE TOPOLOGY

The results of the performance evaluation is shown in Figure 4. Varying the number of HTTP requests per second from 10 to 200, the average response time grows from 295 ms to 500 ms; while the minimum value remains essentially stable. Finally, a slight growth is shown for the maximum value.

Instead, the results considering the scenario as a whole are shown in Figure 5. The trend is similar to the case of single HTTP requests.

A. Execution Environment Setup

With this scenario we test the setup of an execution environment considering the insertion of a new entry and the related synchronization with the LCP running on it.

The results of the evaluation are shown in Figure 6. The average response time varies from 300 ms to more than 700 ms. Unlike the previous scenario, there is a slight growth even for the minimum value. The other statistics follow more or less the same trend of the average case with similar consideration.

[1]https://github.com/astrid-project/astrid-framework

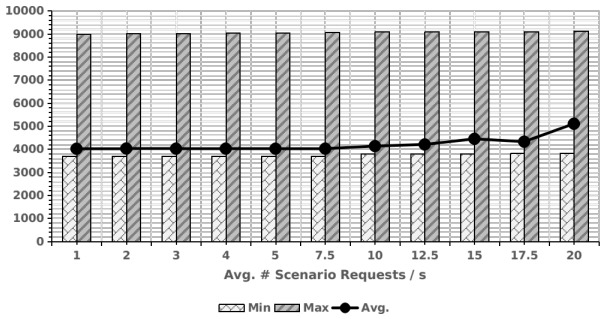

Figure 5. Response Time (in ms) varying the number of the HTTP requests grouped by scenario for the service topology case.

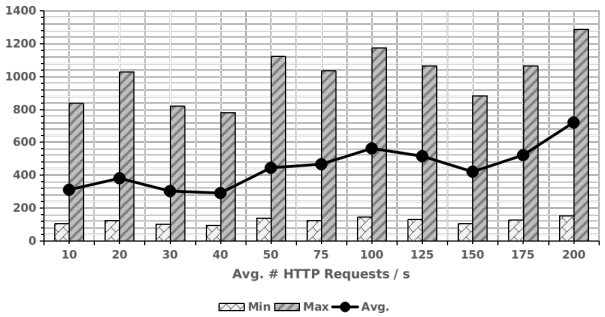

Figure 6. Response Time (in ms) varying the number of the single HTTP requests for the execution environment setup scenario.

Instead, for the results considering the grouped HTTP requests, the difference in the average value as the number of requests increases is less significant (Figure 7). It is even possible to notice a trend that is not totally growing but more undulatory. The other statistics also exhibit these characteristics. Most likely one of the reasons for this variability is due to the synchronization part of the LCP that is managed by the CB-Man with periodic polling.

Finally, Table III shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the insertion of a new execution environment (POST method); while the faster are the ones related to read the info about the execution environment (type and exec-env with GET method).

VII. AGENT MANAGEMENT

This section includes the two scenarios related to the agent management case. The first one is used to evaluate the execution of the start and stop actions for an agent instance; while the second one to evaluate the update of a parameter of an agent instance.

VIII. ACTION

The results are shown in Figure 8. The average response time grows from less of 400 ms to more than 600 ms varying the number of HTTP requests from 10 to 200. Instead, the minimum statistic is more or less constant. Highly variable is the trend of the maximum value. As in the previous scenario, there are very fluctuating values due to the execution of the actions by the LCP and the related interaction with the agent running on the execution environment. The variable values are even more evident when you consider HTTP requests grouped by scenario as shown in Figure 9.

Finally, Table IV shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the insertion of a new agent instance

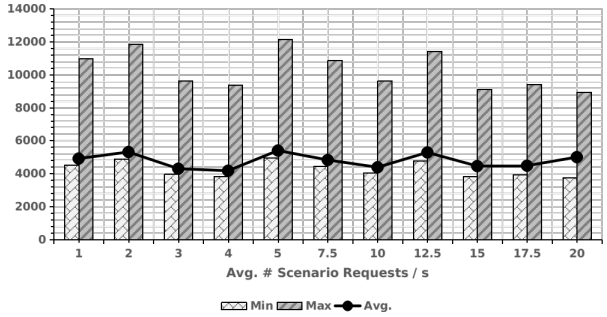

Figure 7. Response Time (in ms) varying the number of the HTTP requests grouped by scenario for the execution environment setup case.

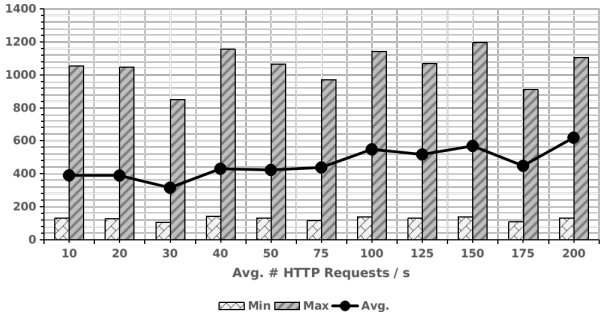

Figure 8. Response Time (in ms) varying the number of the single HTTP requests for the agent management (action) scenario.

TABLE III
PERFORMANCE EVALUATION OF THE EXECUTION ENVIRONMENT SETUP CASE CONSIDERING THE HTTP REQUESTS GROUPED BY TYPE/ENDPOINT WITH 200 AVERAGE # HTTP REQUESTS.

REQUEST		RESPONSE TIME [ms]				
METHOD	END-POINT	AVG	MIN	MAX	STDEV	MEDIAN
GET	/type/exec-env	358	152	384	215	350
POST	/exec-env	1201	510	1289	723	1175
GET	/exec-env	425	180	456	256	416
DELETE	/exec-env	890	377	955	536	871
	TOTAL	**719**	**152**	**1289**	**432**	**703**

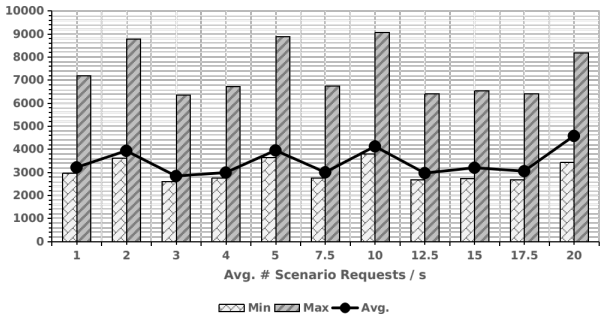

Figure 9. Response Time (in ms) varying the number of the HTTP requests grouped by scenario for the agent management (action) case.

(POST method); while the faster are the ones related to read the info about the execution environment (GET method).

IX. PARAMETER

In the last scenario, we tested the update of a parameter of an agent previously inserted in the catalog and the relative instance.

The results are shown in Figure 10 show a similar trend of the previous scenario. Also in this case, the high variability is due to the interaction with the LCP and the agent instance deployed in the execution environment. Similar consideration for the case considering the HTTP requests grouped by scenario as shown in Figure 11.

Finally, Table V shows the results with 200 number of HTTP requests grouped by type and HTTP endpoint. The most expansive requests are the ones related to the insertion of a new agent instance (POST method); while the faster are the ones related to read the info about the execution environment (GET method).

X. CONCLUSIONS

In this paper, we present a novel framework for the management of a multi-tier architecture, where a common, programmable, and pervasive context fabric feeds a powerful set of multi-vendor detection and analysis algorithms (business logic). The CB is the logical component to manage the security context. We define the security context as the set of information, data, and measurements that describe the service and can be used for security-related purposes. The LCP gives the CB access to the configuration of agents. We performed an evaluation of the CB-Man and LCP considering different scenarios and workloads. The goal is to verify the robustness and the reliance of these two components in different execution scenarios. The tests done show a good behavior of the CB-Man and LCP. The two components are able to satisfy a significant number of requests without loss and with a good response time.

This paper focuses exclusively on evaluating the performance, robustness, and reliability of the two proposed components. Our analysis provides insights into the practical feasibility and effectiveness of these components in real-world applications. However, to ensure comprehensive security, future works will delve into the specific security implications and vulnerabilities associated with these components. By conducting a thorough security analysis, we aim to identify potential threats and develop mitigation strategies to safeguard the integrity and confidentiality of sensitive data.

ACKNOWLEDGMENT

This research was supported supported by the Horizon european project HORSE (grant agreement no. 101096342).

REFERENCES

[1] A. Carrega and M. Repetto, "Data Log Management for Cyber-Security Programmability of Cloud Services and Applications," in *Proceedings of the 1st ACM Workshop on Workshop on Cyber-Security Arms Race*, ser. CYSARM'19, event-place: London, United Kingdom, New York, NY, USA: Association for Computing Machinery, 2019, pp. 47–52, ISBN: 978-1-4503-6840-7. DOI: 10 . 1145 / 3338511 . 3357351. [Online]. Available: https://doi.org/10.1145/3338511.3357351.

[2] A. Mtibaa, K. A. Harras, and H. Alnuweiri, "Malicious Attacks in Mobile Device Clouds: A Data Driven Risk Assessment," *2014 23rd International Conference on Computer Communication and Networks (ICCCN)*, pp. 1–8, Aug. 2014. DOI: 10.1109/ICCCN.2014.6911812.

[3] J. A. Parra, S. A. Gutiérrez, and J. W. Branch, "A Method Based on Deep Learning for the Detection and Characterization of Cybersecurity Incidents in Internet of Things Devices," *arXiv*, Mar. 2022. eprint: 2203.00608.

[4] Elastic, *The Elastic Stack*. [Online]. Available: https://www.elastic.co/elk-stacks (visited on 03/17/2022).

TABLE IV
PERFORMANCE EVALUATION OF THE AGENT MANAGEMENT (ACTION) CASE CONSIDERING THE HTTP REQUESTS GROUPED BY TYPE/ENDPOINT WITH 200 AVERAGE # HTTP REQUESTS.

REQUESTS		RESPONSE TIME [ms]				
METHOD	END-POINT	AVG	MIN	MAX	STDEV	MEDIAN
GET	/exec-env	201	131	222	121	197
POST	/catalog/agent	654	426	723	393	641
POST	/instance/agent	1001	652	1106	602	981
GET	/instance/agent	423	276	467	254	414
PUT	/instance/agent	696	454	769	418	682
DELETE	/instance/agent	648	422	716	389	635
DELETE	/catalog/agent	694	452	767	417	680
	TOTAL	**617**	**131**	**1106**	**371**	**604**

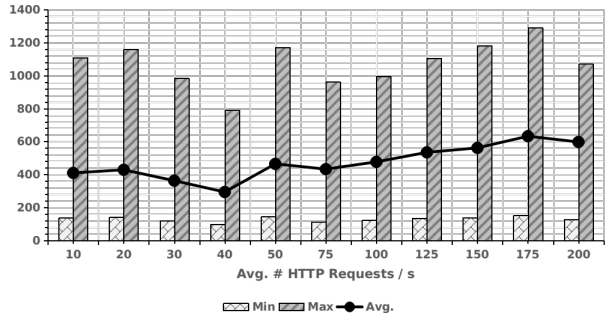

Figure 10. Response Time (in ms) varying the number of the single HTTP requests for the agent management scenario.

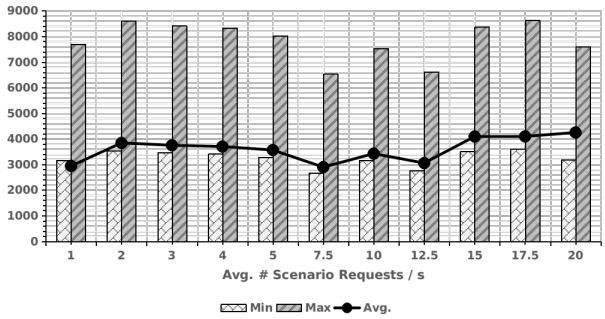

Figure 11. Response Time (in ms) varying the number of the HTTP requests grouped by scenario for the agent management case.

TABLE V
PERFORMANCE EVALUATION OF THE AGENT MANAGEMENT CASE CONSIDERING THE HTTP REQUESTS GROUPED BY ENDPOINT WITH 200 AVERAGE # HTTP REQUESTS.

REQUEST		RESPONSE TIME [ms]				
METHOD	END-POINT	AVG	MIN	MAX	STDEV	MEDIAN
GET	/exec-env	241	126	266	145	236
POST	/catalog/agent	675	353	745	405	660
POST	/instance/agent	970	507	1071	583	949
GET	/instance/agent	457	239	505	274	447
PUT	/instance/agent	690	361	762	414	675
DELETE	/instance/agent	569	297	628	342	557
DELETE	/catalog/agent	577	302	637	346	565
	TOTAL	**597**	**126**	**1071**	**359**	**584**

[5] Elastic, *Elasticsearch*. [Online]. Available: https://www.elastic.co/elasticsearch (visited on 03/17/2022).

[6] Elastic, *Logstash*. [Online]. Available: https://www.elastic.co/logstash (visited on 03/17/2022).

[7] Elastic, *Kibana*, Jan. 2022. [Online]. Available: https://www.elastic.co/kibana (visited on 04/2022).

[8] H. Harkous, C. Papagianni, K. D. Schepper, *et al.*, "Virtual Queues for P4: A Poor Man's Programmable Traffic Manager," *IEEE Transactions on Network and Service Management*, vol. 18, no. 3, pp. 2860–2872, Sep. 2021, ISSN: 1932-4537. DOI: 10.1109/TNSM.2021.3077051.

[9] E. Rios, W. Mallouli, M. Rak, *et al.*, "SLA-driven Monitoring of Multi-Cloud Application Components Using the MUSA framework," *2016 IEEE 36th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, pp. 55–60, Jun. 2016. DOI: 10.1109/ICDCSW.2016.29.

[10] D. C. Erdil, "Simulating peer-to-peer cloud resource scheduling," *Peer-to-Peer Networking and Applications*, vol. 5, no. 3, pp. 219–230, Sep. 2012, ISSN: 1936-6442. DOI: 10.1007/s12083-011-0112-8.

[11] S. Jeong, H. Kim, J.-H. Yoo, *et al.*, "Machine Learning based Link State Aware Service Function Chaining," *2019 20th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, vol. 00, pp. 1–4, Jan. 2019. DOI: 10.23919/APNOMS.2019.8893037.

[12] M. Repetto, D. Striccoli, G. Piro, *et al.*, "An Autonomous Cybersecurity Framework for Next-generation Digital Service Chains," *Journal of Network and Systems Management*, vol. 29, no. 4, p. 37, Oct. 2021, ISSN: 1064-7570. DOI: 10.1007/s10922-021-09607-7.

[13] R. Pugliese, S. Regondi, and R. Marini, "Machine learning-based approach: Global trends, research directions, and regulatory standpoints," *Data Science and Management*, Dec. 2021, ISSN: 2666-7649. DOI: 10.1016/j.dsm.2021.12.002.

[14] M. Fukuda, K. Kashiwagi, and S. Kobayashi, "AgentTeamwork: Coordinating grid-computing jobs with mobile agents," *Applied Intelligence*, vol. 25, no. 2, p. 181, Oct. 2006, ISSN: 0924-669X. DOI: 10.1007/s10489-006-9653-6.

[15] P. Network, *Polycube*. [Online]. Available: https://github.com/polycube-network/polycube (visited on 03/17/2022).